

Lista de exercícios — Árvore Binária de Busca

Estrutura base em C:

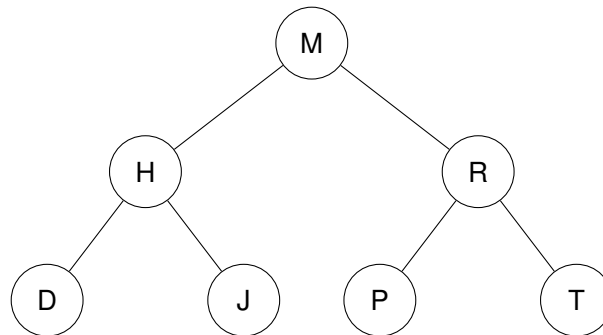
```
typedef struct NoArv {  
    int valor;  
    struct NoArv* esq;  
    struct NoArv* dir;  
} NoArv;
```

Observações:

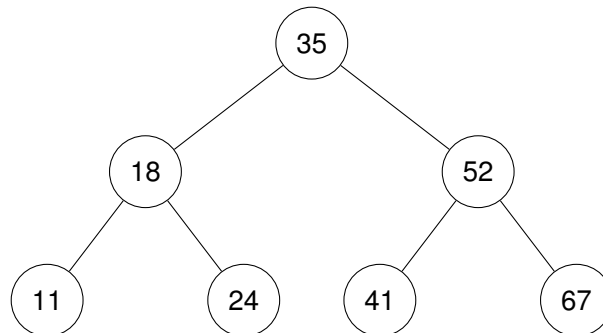
- Nesta lista, quando a remoção de um nó com dois filhos for necessária, utilize o **sucessor em-order**, salvo indicação em contrário.
- Quando for pedido para desenhar a árvore, represente claramente os filhos esquerdo e direito.
- Nas questões de implementação, considere que a ABB não possui valores repetidos.

1. Em uma Árvore Binária de Busca (ABB), assinale a alternativa correta.
 - (a) Todo nó da subárvore esquerda possui valor maior que o da raiz.
 - (b) Todo nó da subárvore direita possui valor menor que o da raiz.
 - (c) Todo nó da subárvore esquerda possui valor menor que o da raiz, e todo nó da subárvore direita possui valor maior que o da raiz.
 - (d) Uma ABB pode ter qualquer organização, desde que a raiz seja o menor elemento.
2. Em uma estrutura de dados do tipo árvore, o **grau de um nó** refere-se:
 - (a) ao número de filhos que esse nó possui.
 - (b) ao maior caminho desse nó até a folha mais distante.
 - (c) à profundidade desse nó.
 - (d) ao número total de descendentes desse nó.
3. Dada a árvore abaixo, indique:

- (a) os nós folha;
- (b) o grau da árvore;
- (c) a altura da árvore;
- (d) os descendentes do nó H.

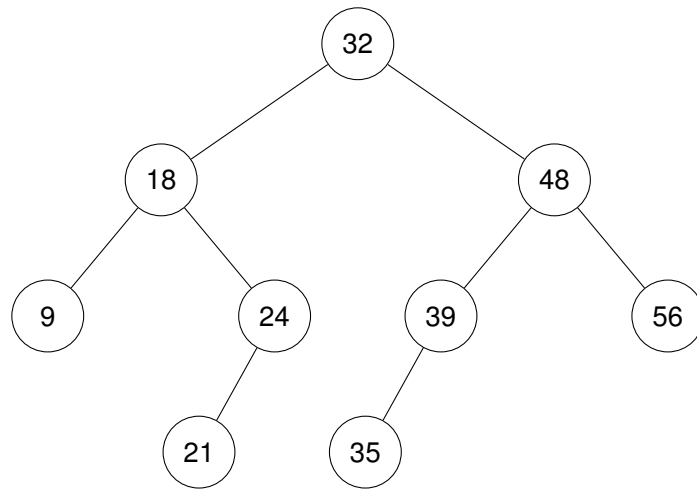


4. Considere a árvore binária abaixo.



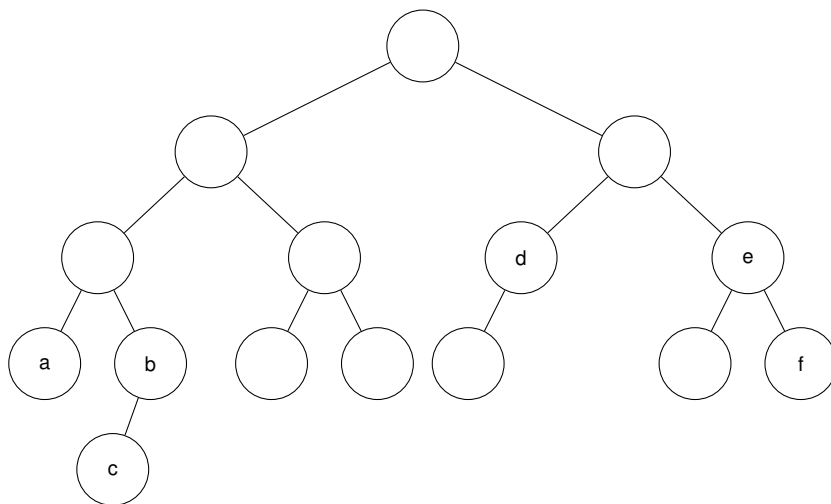
Escreva as sequências de visita para:

- (a) pré-ordem;
 - (b) em ordem;
 - (c) pós-ordem.
5. Dada uma ABB inicialmente vazia, faça as seguintes inserções e **desenhe a árvore resultante**:
- 45, 20, 60, 10, 30, 50, 70, 25, 33
6. Suponha que a árvore abaixo seja percorrida em **em-ordem** e os valores visitados sejam inseridos, nessa ordem, em uma estrutura do tipo fila.



Desenhe a fila resultante.

7. A figura representa apenas a **estrutura** de uma ABB com 15 nós distintos. As letras servem apenas para identificar alguns nós.



Assinale a alternativa que apresenta, respectivamente, os nós que armazenam o **segundo menor** e o **segundo maior** valores da árvore.

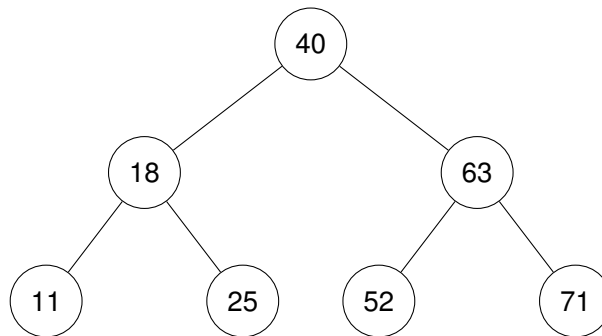
- (a) b – f
- (b) a – e
- (c) b – d
- (d) c – f
- (e) a – d

8. Classifique cada afirmação em **V** ou **F**.

- I. A travessia em ordem de uma ABB produz os valores em ordem crescente.
- II. Todo nó folha de uma ABB possui exatamente dois filhos nulos.
- III. Uma ABB pode possuir valores repetidos sem alterar sua definição.

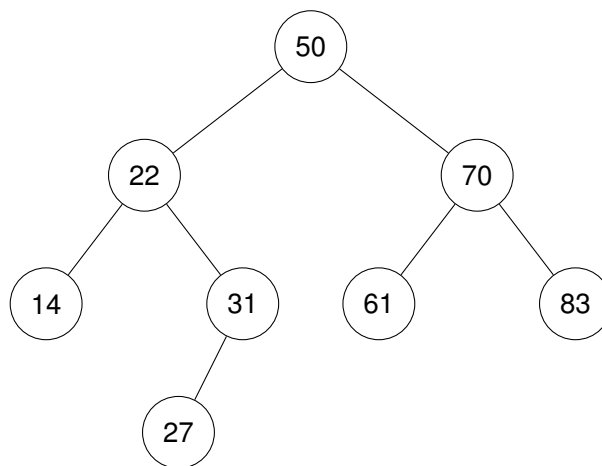
IV. Em uma ABB, buscar um valor menor que a raiz implica seguir para a subárvore esquerda.

9. Considere a ABB abaixo.



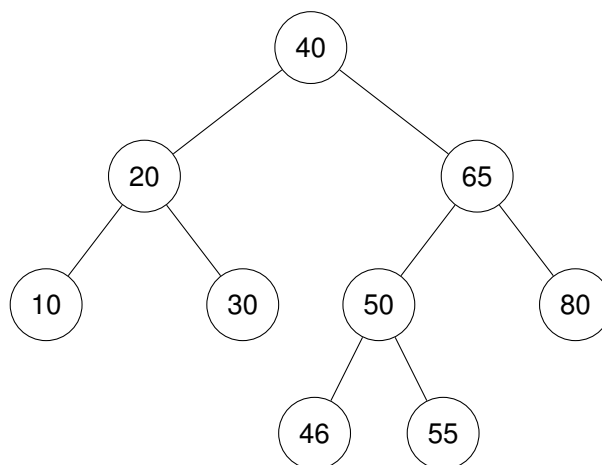
Desenhe a árvore após a remoção do valor 11.

10. Considere a ABB abaixo.



Desenhe a árvore após a remoção do valor 31.

11. Considere a ABB abaixo e remova o valor 40. Em seguida, desenhe a árvore resultante.



12. Classifique cada afirmação em **V** ou **F**.

I. Em uma ABB, todo nó da subárvore esquerda de um nó possui valor menor que o valor desse nó.

- II. Se a travessia em ordem de uma árvore produz uma sequência crescente, então essa árvore é uma ABB válida.
- III. O menor valor de uma ABB sempre pode ser encontrado seguindo-se os ponteiros para a esquerda até onde for possível.
- IV. Em uma ABB sem valores repetidos, o sucessor em ordem de um nó é sempre o seu filho direito.
- V. Uma árvore com apenas um nó é, ao mesmo tempo, uma ABB válida, cheia e balanceada.

13. Implemente a função abaixo, que retorna um ponteiro para o nó que contém `valor`, ou `NULL` caso ele não esteja na árvore.

```
NoArv* buscarABB(NoArv* raiz, int valor);
```

14. Implemente a função abaixo.

```
int contarNos(NoArv* raiz);
```

15. Implemente a função abaixo, que retorna a altura da árvore.

```
int altura(NoArv* raiz);
```

16. Implemente uma função que retorne a quantidade de nós folha da árvore.

```
int contarFolhas(NoArv* raiz);
```

17. Implemente uma função que imprima os valores da ABB em ordem crescente.

```
void mostrarEmOrdem(NoArv* raiz);
```

18. Implemente uma função que retorne o **menor valor** armazenado em uma ABB não vazia.

```
int menorABB(NoArv* raiz);
```

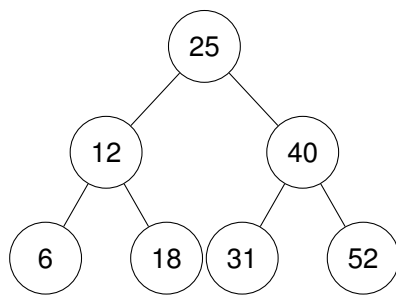
19. Faça uma função que **espelhe** uma árvore binária, trocando recursivamente as subárvores esquerda e direita de todos os nós.

```
void espelhaArvore(NoArv* raiz);
```

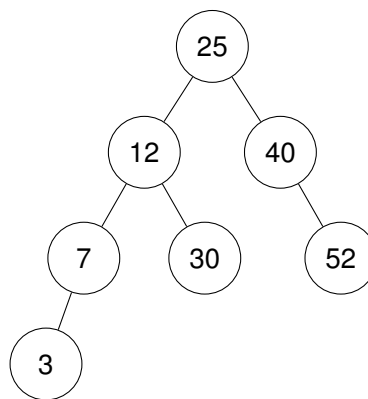
20. Considere as três árvores abaixo e, para **cada uma delas**, informe:

- (a) se a árvore é **completa** ou **incompleta**;
- (b) se a árvore é **cheia** ou **não cheia**;
- (c) se a árvore é **balanceada** ou **não balanceada**;
- (d) o **grau da árvore**;
- (e) se a árvore é uma **ABB válida** ou **ABB inválida**.

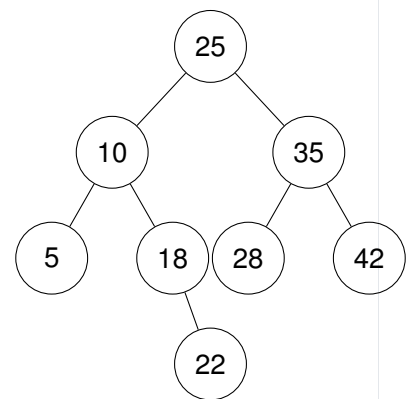
Justifique brevemente cada classificação.



Árvore A



Árvore B



Árvore C

21. Faça as seguintes inserções e remoções em uma árvore binária de busca, originalmente vazia.

Inserir:

24, 66, 71, 13, 17, 23, 49, 31, 37, 82, 11

Remover:

82, 13, 23, 49

Desenhe a árvore após **todas** as operações.

22. Uma ABB inicialmente vazia recebe as operações abaixo, nesta ordem:

Inserir 50, Inserir 30, Inserir 70, Inserir 20, Inserir 40, Inserir 60,
Remover 30, Inserir 80, Inserir 65, Inserir 90, Inserir 75, Remover 70

A remoção de um nó com dois filhos deve seguir a estratégia do **sucessor em-order**. Desenhe a árvore resultante ao final de todas as operações.

23. Considere uma árvore binária de busca inicialmente vazia. Insira, **nessa ordem**, os seguintes elementos:

M, F, S, D, J, P, U, A, E, H, Q, T, W, K

Desenhe a árvore resultante.

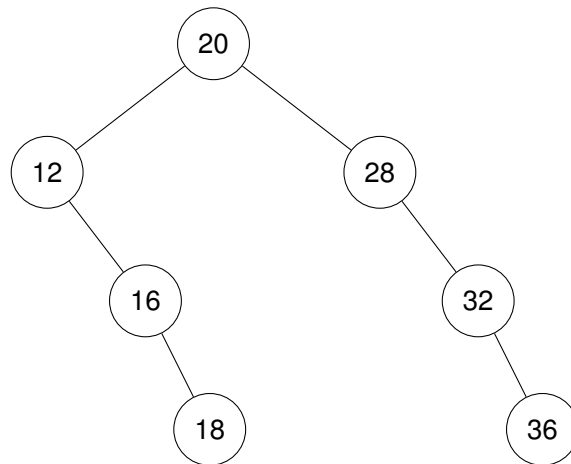
24. Considere a seguinte sequência de inserção em uma ABB inicialmente vazia:

48, 26, 12, 37, 61, 55, 74, 52, 58

Responda:

- (a) Qual será a raiz da árvore?
- (b) Quem será o pai do nó 58?
- (c) O nó 37 será filho esquerdo ou direito de qual nó?
- (d) Quantos nós folha existirão ao final?

25. Observe a estrutura abaixo.



Assinale a alternativa correta.

- (a) A árvore é uma ABB válida; se o valor 17 fosse inserido, ele ficaria à esquerda do nó 16.
- (b) A árvore é uma ABB válida; se o valor 22 fosse inserido, ele ficaria diretamente à direita do nó 20.
- (c) A árvore é uma ABB válida; se o valor 14 fosse inserido, ele ficaria à esquerda do nó 16.
- (d) A árvore é uma ABB inválida, pois o valor 18 não poderia estar na subárvore de 12.
- (e) A árvore é uma ABB desbalanceada e inválida.

26. Explique por que a travessia **em ordem** de uma ABB devolve os valores em ordem crescente.

27. Escreva uma função que percorra a ABB e conte quantos nós armazenam valores **pares**. Ao final, a função deve **retornar a quantidade total** de nós cujo campo `info` é divisível por 2.

```
int contarPares(NoArv* raiz);
```

28. Dadas duas ABBs, dizemos que elas são **iguais** se:

- ambas forem vazias; ou
- ambas possuírem uma raiz com o mesmo valor, e além disso suas subárvores esquerdas forem iguais e suas subárvores direitas também forem iguais.

Escreva uma função que compare duas árvores e **retorne 1** caso elas sejam iguais nessa definição, ou **retorne 0** caso contrário.

```
int abbIguais(NoArv* a1, NoArv* a2);
```

29. Uma ABB é dita **cheia** quando todo nó possui **zero** ou **dois filhos**, ou seja, nenhum nó pode ter apenas um filho.

Escreva uma função que verifique essa propriedade e:

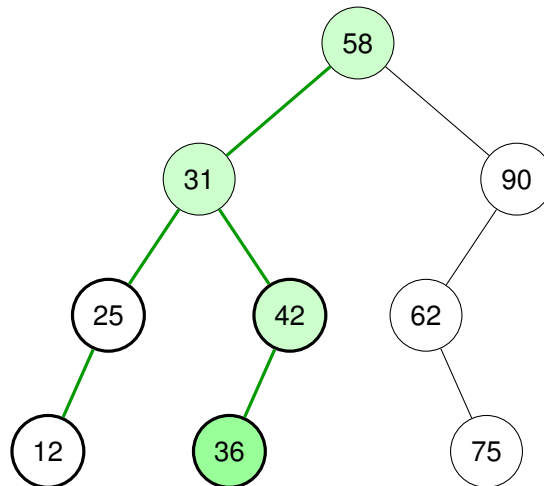
- **retorne 1**, se a árvore for estritamente binária;
- **retorne 0**, caso exista pelo menos um nó com apenas um filho.

```
int ehCheia(NoArv* raiz);
```

30. Uma empresa de logística organiza seus centros de distribuição em uma estrutura hierárquica para facilitar a busca por uma unidade específica. Cada centro possui um código numérico, e a organização segue a lógica de uma Árvore Binária de Busca: códigos menores ficam à esquerda e códigos maiores à direita.

Quando se deseja localizar um centro, o sistema deve exibir por quais centros é necessário passar, a partir da raiz, até chegar ao código procurado.

Considere a árvore abaixo, em que está destacado o caminho até o centro de código 36:



Implemente a função abaixo, que deve mostrar os valores do caminho da raiz até o valor procurado, caso ele exista na árvore.

```
int mostraCaminho(NoArv* raiz, int valor);
```

Por exemplo, para o valor 36, a saída esperada é:

58 31 42 36

Sugestão: a função pode retornar 1 quando encontra o valor e 0 caso contrário.